🔍 AUDITORIA TÉCNICA COMPLETA — VIVA360

Data: 19/02/2026 | Versão: 1.1.0

📊 **NOTA GERAL: 7.0 / 10**

		🟢
Segurança	7.0	🟡 Bom com gaps
		🔴
Qualidade de Código	7.5	🟡 Bom
		🟡
Performance	8.0	🟢 Bom
		🟡
UX/UI	7.5	🟡 Bom

---

1. VISÃO GERAL DO PROJETO

O Viva360 é um ecossistema holístico full-stack com três perfis de usuário (Buscador/Cliente, Guardião/Profissional, Santuário/Espaço). A stack utilizada é **React 19 + TypeScript + Vite** no frontend e **Express + Prisma + Supabase** no backend, deployado no Vercel com banco PostgreSQL.

Stack Completa Identificada

- **Frontend**: React 19, TypeScript, Vite 6, TailwindCSS, Framer Motion, PWA
- **Backend**: Node.js, Express 4, TypeScript, Prisma 6, Supabase Auth
- **Banco**: PostgreSQL via Supabase (30+ modelos no schema)
- **Infra**: Vercel (frontend + serverless), Prometheus, BullMQ, Redis, Cloudinary
- **Testes**: Playwright (E2E), Vitest (unit), stress tests customizados

---

2. PROBLEMAS CRÍTICOS 🔴

2.1 — 107 Telas sem Implementação (Bloqueante)

O `AUDIT_RESULTS.json` detectou **107 estados de FlowEngine sem componente React mapeado**:

- **BUSCADOR**: 41 estados sem tela (ex: `PAYMENT_HISTORY`, `GARDEN_VIEW`, `EVO_GRIMOIRE`, `METAMORPHOSIS_RITUAL`, `TRIBE_INVITE`, `CHAT_ROOM`)
- **GUARDIÃO**: 30 estados sem tela
- **SANTUÁRIO**: 36 estados sem tela

Impacto: Qualquer navegação que leve a esses estados exibe a tela de fallback "Sintonização Necessária" em vez de uma funcionalidade real. Isto representa metade das jornadas de usuário **quebradas**.

Solução: Mapear cada estado em `screenMap.tsx` para um componente existente ou criar as telas. Muitos estados provavelmente reutilizam telas existentes mas estão com nomes inconsistentes entre FlowEngine e screenMap.

2.2 — Endpoint de Debug Exposto em Produção (Segurança Crítica)

Arquivo: `backend/src/app.ts`, linha 109

```
POST /api/debug-login
```

Este endpoint não exige autenticação e expõe stack traces completos em produção:

```
app.post('/api/debug-login', async (req, res) => {
    // ...
    res.status(500).json({
        error: err.message,
        name: err.name,
        stack: err.stack?.split('\n').slice(0, 5), // ⚠️ VAZA STACK TRACE
```

Solução imediata: Remover ou proteger com autenticação admin + bloquear em `NODE_ENV === 'production'`.

2.3 — Telemetria e Guarda de Produção Desativados

Arquivo: `backend/src/app.ts`

```
// initTelemetry();           // ← COMENTADO
// assertCriticalProdConfig(); // ← COMENTADO
```

Esses guards foram desabilitados durante debugging e nunca reabilitados. Em produção, a app pode iniciar sem variáveis críticas configuradas, causando falhas silenciosas.

---

3. PROBLEMAS DE ALTA PRIORIDADE 🟠

3.1 — Checkout Flow sem Tratamento de Erros (7 componentes)

O `AUDIT_RESULTS.json` identificou 7 componentes de pagamento sem estados adequados:

Arquivo	Problemas
`BookingSearch.tsx`	Sem tratamento de erro
`BookingSelect.tsx`	Sem loading, sem erro, sem feedback de sucesso
`BookingConfirm.tsx`	Sem loading state
`Checkout.tsx`	Sem loading, sem erro, sem feedback

`PaymentSuccess.tsx`	Sem loading, sem erro
`PaymentHistoryScreen.tsx`	Sem feedback de sucesso
`components/Checkout.tsx`	Sem loading, sem erro

Impacto: Transações financeiras sem feedback adequado = usuários perdidos, duplo-clique em pagamentos, experiência péssima.

3.2 — `reviews.routes.ts` Não Registrado

O arquivo `backend/src/routes/reviews.routes.ts` existe, o controller `reviews.controller.ts` existe e o modelo `Review` no Prisma existe, mas a rota **nunca foi importada/registrada** em `backend/src/routes/index.ts`.

Impacto: Sistema de avaliações completamente inoperante via API.

Fix:

```
// Adicionar em index.ts:
import reviewsRoutes from './reviews.routes';
router.use('/reviews', authenticateUser, reviewsRoutes);
```

3.3 — Token Mock Hardcoded no Frontend

Arquivo: `services/api.ts`, linha 11

```
const MOCK_AUTH_TOKEN = 'admin-excellence-2026';
```

Embora limitado a modo MOCK, este token está comprometido (commitado no GitHub público). Deve ser movido para variável de ambiente ou rotacionado.

3.4 — Ícones PWA de CDN Externo

Arquivo: `vite.config.ts`

```
src: 'https://cdn-icons-png.flaticon.com/512/3209/3209931.png',
```

Ícones PWA dependem de CDN de terceiro (Flaticon). Se o CDN estiver indisponível, o app não instala corretamente. Além disso, usar ícones sem licença pode ser problema legal.

Solução: Criar ícones próprios e hospedar em `/public/`.

---

4. PROBLEMAS DE MÉDIA PRIORIDADE 🟡

4.1 — Gamificação Incompleta

Sistemas prometidos no schema e tipos mas com `usage_count = 0`:

- **Challenges** — definido nos tipos, nunca usado nas views
- **Rank** — definido nos tipos, nunca usado nas views

- **Leaderboard** — uso_count = 1 (presente mas não funcional)

4.2 — Elementos Ocultos com onClick (3 arquivos)

`ProDashboard.tsx`, `ClientDashboard.tsx`, `SmartTutorial.tsx` contêm elementos com handlers de clique marcados como hidden. São dead code ou acessibilidade quebrada.

4.3 — 6 Telas Estáticas sem Navegação

`RitualsView.tsx`, `PaymentHistoryScreen.tsx`, `ClientMarketplace.tsx` e outros têm 0 navegações saindo delas — o usuário fica "preso".

4.4 — `getProgress()` Hardcoded no FlowEngine

```
public getProgress(): number {
    return this.history.length * 5; // ← Placeholder, não reflete progresso real
}
```

4.5 — Falta `connection_limit` no Prisma para Serverless

Para Vercel serverless, o Prisma precisa de configuração especial:

```
?connection_limit=1&pgbouncer=true
```

Sem isso, cada invocação serverless pode criar novas conexões, esgotando o pool do Supabase rapidamente.

---

5. O QUE ESTÁ BEM ✅

5.1 — Arquitetura FlowEngine (Excelente)

O padrão State Machine por perfil (`BuscadorFlowEngine`, `GuardiaoFlowEngine`, `SantuarioFlowEngine`) é uma escolha arquitetural sofisticada e correta para um SPA com múltiplos fluxos complexos. A abstração `baseFlow.ts` e o `ScreenConnector` são elegantes.

5.2 — Schema Prisma Abrangente e Bem Modelado

30+ modelos cobrindo: agendamentos, chat, gamificação, oracle, contratos, recrutamento, marketplace, escambo, registros clínicos, audit events. Relações bem definidas com foreign keys e índices.

5.3 — Camadas de Segurança Sólidas

- Helmet com CSP configurável
- CORS com allowlist de origens
- Rate limiting global + middleware específico
- Circuit Breaker implementado
- WAF middleware (`security.middleware.ts`)
- `securityHardening` com headers OWASP

5.4 — Infraestrutura de Performance

- Code splitting manual por perfil (buscador/guardião/santuário são chunks separados)
- Gzip/Brotli via `compression`

- SWR Cache middleware para tribe e marketplace
- `lazyWithRetry` para imports com retry automático
- Prometheus metrics em `/metrics` com auth

5.5 — Auth Dual-Path Robusto

- Frontend: Supabase Auth direto
- Backend: Verificação via `supabaseAdmin.auth.getUser()` + fallback JWT
- Auto-criação de perfil no login
- Suporte a multi-role com `profile_roles`
- Allowlist para controle de acesso

5.6 — PWA Configurado

Manifest com `display: standalone`, `orientation: portrait`, `theme_color` e `autoUpdate`. App pode ser instalado.

5.7 — Infraestrutura de Testes Rica

- Playwright E2E com configs separadas (smoke, QA, audit)
- Vitest para testes unitários backend (auth, checkout, security)
- Scripts de stress test enterprise (5K usuários)
- Chaos engineering middleware (`chaos_monkey.sh`)

---

6. ANÁLISE DE ARQUIVO-CHAVE: `services/api.ts` (1.808 linhas)

Este arquivo é o maior ponto de dívida técnica do frontend. Com 1.808 linhas, combina:

- Lógica de autenticação (login, logout, OAuth, register)
- Comunicação com todos os endpoints do backend
- Gerenciamento de sessão (mock + real)
- Normalização de dados

Recomendação: Dividir em módulos menores: `auth.service.ts`, `profile.service.ts`, `booking.service.ts`, etc. O tamanho atual dificulta manutenção e testes.

---

7. ROADMAP DE CORREÇÕES PRIORIZADAS

Semana 1 — Crítico (Para produção sadia)

1. **Remover** `POST /api/debug-login` do app.ts
2. **Reabilitar** `assertCriticalProdConfig()` com tratamento correto
3. **Mapear** estados faltantes no screenMap.tsx (muitos já existem como telas, só o mapeamento está errado)
4. **Registrar** `reviews.routes.ts` em index.ts

Semana 2 — Alto impacto

5. **Adicionar** loading states e error handling nos 7 componentes de checkout
6. **Adicionar** `connection_limit=1&pgbouncer=true` na string de conexão Prisma
7. **Criar** ícones PWA próprios e self-hosted
8. **Rotacionar** `MOCK_AUTH_TOKEN` para variável de ambiente

Semana 3 — Médio impacto

9. **Implementar** challenges e rank no frontend (modelos já existem no banco)
10. **Adicionar** navegação de saída nas 6 telas estáticas
11. **Corrigir** `getProgress()` com lógica real de progresso
12. **Dividir** `services/api.ts` em módulos especializados

Semana 4 — Polimento

13. **Remover** `console.log` do backend (já desativado no build frontend via Terser)
14. **Limpar** arquivos orphaned de teste na raiz do projeto
15. **Documentar** sistema de FlowEngine para novos desenvolvedores

---

8. POTENCIAL REAL DO PROJETO

O Viva360 tem uma base técnica **excepcional** para um projeto de produto digital. A arquitetura FlowEngine, a separação de concerns, o schema de banco abrangente e as camadas de infraestrutura mostram um nível de maturidade técnica acima da média.

O gap entre a nota atual (7.0) e o potencial (9.0+) é quase inteiramente causado por **implementação incompleta de telas** — problema de volume de trabalho, não de arquitetura.

Com as correções da Semana 1, o projeto alcança **7.8/10** e pode ser considerado produção-ready para beta. Com todas as correções: **8.5 — 9.0 / 10**.

Auditoria realizada por Claude (Anthropic) — Análise estática de código, schema, rotas e configurações.
Data: 19/02/2026